

Mapeamento de Comunicação entre Serviços

10 Fluxos REST	7 Microserviços	Fase 1 Sem RabbitMQ
----------------	-----------------	---------------------

Escopo: Este documento detalha todos os fluxos de comunicação REST síncrona entre os microserviços do MatchPass. RabbitMQ e comunicação assíncrona serão adicionados na fase final do projeto e não estão cobertos aqui.

Índice de Fluxos

F1	Criação de assentos no inventário	Event Catalog
F2	Autenticação e validação de JWT	Identity
F3	Listagem e detalhe de eventos	Event Catalog
F4	Reserva temporária de assento (lock de 10 min)	Inventory
F5	Criação do pedido	Checkout/Order
F6	Processamento de pagamento	Payment
F7	Emissão do ingresso digital	Ticket
F8	Validação de acesso na catraca	Ticket
F9	Cancelamento de pedido e estorno	Checkout/Order
F10	Roteamento e controle de acesso	API Gateway

F1**Criação de assentos no inventário**

Inventory Service consulta o Event Catalog antes de inicializar os locks

Event Catalog

Antes de qualquer venda, o Inventory Service precisa saber quantos assentos existem por setor e se são numerados. Este fluxo popula o Redis com os registros iniciais de disponibilidade. Já implementado.

Passo 1 Verificar existência e status do evento

Inventory Service

→ Event Catalog

GET

GET /events/{eventId}

Validar	Evento existe (HTTP 404 → abortar)
Validar	event.status == SCHEDULED (CANCELED/FINISHED → abortar)
Retorna	EventDetailsResponse: eventId, title, eventDate, venueId

Response: EventDetailsResponse { eventId, title, eventDate, venue, availableSectors }**Passo 2 Obter capacidade e configuração do setor**

Inventory Service

→ Event Catalog

GETGET
/events/{eventId}/sectors/{sectorId}

Validar	Setor pertence ao evento
Validar	sector.capacity > 0
Obter	hasNumberedSeats: define se lockId usa seatNumber ou quantity

Response: SectorPricingResponse { sectorId, sectorName, basePrice, hasNumberedSeats }**Passo 3 Inicializar registros de disponibilidade no Redis**

Inventory Service

→ Redis

WRITE

RedisHash: SeatLock (N registros)

Ação	Para setor numerado: criar 1 SeatLock por assento com status AVAILABLE
Ação	Para setor geral: criar 1 SeatLock com quantity = capacity, status AVAILABLE
TTL	Sem TTL nesta etapa — expiração só ocorre ao fazer LOCK

Response: N/A – escrita interna no Redis

Nota: Pré-condição para todos os demais fluxos. Deve ser idempotente (re-executar não duplica registros).

Casos de Erro

Evento não encontrado	HTTP 404 → lançar exceção, não inicializar nada
Evento cancelado/encerrado	HTTP 422 → retornar erro ao chamador
Setor sem capacidade	HTTP 422 → ignorar o setor

F2

Autenticação e validação de JWT

Gate transversal — toda requisição protegida passa por aqui

Identity

O Identity Service emite e valida tokens JWT. O API Gateway intercepta cada requisição e valida o token antes de rotear. UserRole no JWT determina acesso: CUSTOMER, ADMIN ou GATE_OPERATOR.

Passo 1 Login do usuário

Frontend → Identity Service **POST** POST /auth/login

Validar	Email existe no banco (users)
Validar	passwordHash confere com BCrypt
Validar	user.isActive == true (conta não bloqueada)

Request: AuthRequest { email: string, password: string }

Response: AuthTokenResponse { accessToken (JWT), refreshToken, expiresIn (segundos) }

Passo 2 Refresh do token

Frontend → Identity Service **POST** POST /auth/refresh

Validar	refreshToken não expirado e não revogado
Validar	userId do refresh bate com o banco

Request: { refreshToken: string }

Response: AuthTokenResponse { accessToken (novo JWT), refreshToken, expiresIn }

Passo 3 Validação do JWT pelo Gateway em toda requisição

API Gateway → Identity Service **GET** GET /auth/validate

Header	Authorization: Bearer
Validar	Assinatura JWT válida (secret compartilhado)
Validar	Token não expirado (exp claim)
Extraí	userId, role → repassa como headers X-User-Id, X-User-Role para os serviços downstream

Request: Header: Authorization: Bearer {jwt}

Response: HTTP 200 (válido) | HTTP 401 (inválido/expirado)

Passo 4 Registro de novo usuário

Frontend → Identity Service **POST** POST /users/register

Validar	Email único (UNIQUE constraint)
Validar	CPF único e formato válido (11 dígitos)
Validar	Senha com força mínima (regra de negócio)
Ação	Hashear senha com BCrypt antes de persistir

Request: UserRegistrationRequest { fullName, email, password, document (CPF) }

Response: HTTP 201 + { userId, email, role: CUSTOMER }

Nota: Rotas públicas (não precisam de JWT): POST /auth/login, POST /users/register, GET /events. Todas as demais exigem token.

Casos de Erro

Credenciais inválidas	HTTP 401 — não revelar se é email ou senha errada
Conta inativa	HTTP 403 — conta bloqueada
Token expirado	HTTP 401 — frontend deve usar /auth/refresh
Token inválido	HTTP 401 — rejeitar imediatamente

F3 Listagem e detalhe de eventos

Cliente navega pelo catálogo e verifica disponibilidade antes de comprar

Event Catalog

Dois serviços respondem ao cliente durante a navegação: Event Catalog fornece os dados estruturais (o que existe) e Inventory Service fornece o estado real de disponibilidade (o que está livre). São consultados separadamente.

Passo 1 Listar eventos disponíveis

Frontend → Event Catalog (via Gateway) GET GET /events?status=SCHEDULED

Filtrar	Apenas eventos com status SCHEDULED
Opcional	Paginação: ?page=0&size=20
Opcional	Filtros: ?city=SP, ?dateFrom=, ?dateTo=

Request: Query params: status, city, dateFrom, dateTo, page, size

Response: List { eventId, title, eventDate, venueName, venueCity, startingPrice }

Passo 2 Detalhe do evento com setores e preços

Frontend → Event Catalog (via Gateway) GET GET /events/{eventId}

Validar	Evento existe e status == SCHEDULED
Retorna	Venue completo + todos os setores com preços e disponibilidade de venda

Request: Path param: eventId (UUID)

Response: EventDetailsResponse { eventId, title, eventDate, venue: VenueDto, availableSectors: List }

Passo 3 Verificar disponibilidade real de assentos

Frontend → Inventory Service (via Gateway) GET GET /inventory/{eventId}/sectors/{sectorId}/availability

Retorna	Contagem de SeatLocks com status AVAILABLE no Redis
Retorna	Lista de seatNumbers disponíveis (se setor numerado)
Cache	Alta frequência de consulta — considerar cache de curta duração (5-10s)

Request: Path params: eventId, sectorId

Response: { sectorId, availableCount: int, availableSeats: List (se numerado) }

Nota: Não requer autenticação. O startingPrice em EventSummaryResponse deve refletir o menor basePrice entre os setores do evento.

Casos de Erro

Evento não encontrado	HTTP 404
Evento cancelado	HTTP 410 Gone — evento não está mais disponível
Setor sem disponibilidade	availableCount = 0 — frontend deve desabilitar seleção

F4

Reserva temporária de assento (lock de 10 min)

Primeiro passo da compra — bloqueia o assento antes de criar o pedido

Inventory

O lock garante que dois clientes não comprem o mesmo assento. O Checkout Service aciona o Inventory que valida disponibilidade e cria um SeatLock com TTL de 600s no Redis. Após 10 minutos sem pagamento, o assento é liberado automaticamente.

Passo 1 Checkout solicita reserva ao Inventory

Checkout Service

→ Inventory Service

POST

POST /inventory/reserve

Requer	Header X-User-Id (vindo do Gateway após validar JWT)
Validar	eventId e sectorId existem (pode usar cache da chamada anterior)

Request: `SeatReservationRequest { eventId, sectorId, quantity, seatNumber (null se setor geral) }`

Response: `SeatStatusResponse { lockId, status: LOCKED, expiresAt }`

Passo 2 Inventory verifica disponibilidade no Redis

Inventory Service

→ Redis

GET

`findByEventIdAndSectorIdAndSeatNumber(.
..)`

Setor numerado	Buscar SeatLock com seatNumber exato e status AVAILABLE
Setor geral	Buscar lock do setor e confirmar quantity disponível suficiente
Checar	Nenhum lock LOCKED/SOLD para o mesmo assento

Request: N/A — consulta interna ao Redis

Response: Lista de SeatLocks existentes para validação

Passo 3 Criar SeatLock com TTL no Redis

Inventory Service

→ Redis

WRITE

repository.save(SeatLock)

Gerar	lockId = UUID.randomUUID().toString()
Definir	status = LOCKED, ttl = 600 (10 minutos)
Atômico	Verificação + escrita deve ser atômica — usar Lua script ou transação Redis

Request: `SeatLock { lockId, eventId, sectorId, userId, quantity, seatNumber, status: LOCKED, ttl: 600 }`

Response: SeatLock persistido com TTL

Nota: Atomicidade é crítica: use SETNX do Redis ou Lua script para evitar race condition entre dois clientes requisitando o mesmo assento simultaneamente.

Casos de Erro

Assento já LOCKED ou SOLD	HTTP 409 Conflict — retornar { error: 'SEAT_UNAVAILABLE' }
eventId/sectorId não encontrado	HTTP 404
TTL expirado entre verificação e escrita	HTTP 409 — tratar como conflito

F5 Criação do pedido

Pedido formal após lock confirmado
— persiste em PostgreSQL

Checkout/Order

Com o lockId em mãos, o Checkout Service cria o Order em PostgreSQL. Consulta o Event Catalog para calcular o preço correto e armazena os itens do pedido. O Order é criado com status PENDING e aguarda o pagamento.

Passo 1 Cliente envia pedido de checkout

Frontend → Checkout Service (via Gateway) **POST** POST /orders/checkout

Requer	JWT válido — userId extraído do token
Validar	Items não vazios
Validar	TicketType é um valor válido do enum: INTEIRA, MEIA_ESTUDANTE, MEIA_SOLIDARIA

Request: CheckoutRequest { eventId, items: [{ sectorId, ticketType, seatNumber }] }

Response: OrderSummaryResponse { orderId, totalAmount, status: PENDING, paymentUrl }

Passo 2 Verificar lockId ainda válido

Checkout Service → Inventory Service **GET** GET /inventory/locks/{lockId}/status

Validar	SeatLock existe no Redis (não expirou)
Validar	lock.userId == userId do JWT (não é lock de outro usuário)
Validar	lock.status == LOCKED

Request: Path param: lockId

Response: SeatStatusResponse { lockId, status, expiresAt }

Passo 3 Obter preços do Event Catalog

Checkout Service → Event Catalog **GET** GET /events/{eventId}/sectors/{sectorId}/pricing

Obter	basePrice e halfPrice da EventSectorPricing
Calcular	Preço por item: INTEIRA = basePrice, MEIA_* = halfPrice
Calcular	totalAmount = soma de appliedPrice de todos os OrderItems
Validar	isAvailable == true para o setor

Request: Path params: eventId, sectorId

Response: SectorPricingResponse { sectorId, sectorName, basePrice, hasNumberedSeats }

Passo 4 Persistir Order e OrderItems em PostgreSQL

Checkout Service → PostgreSQL (Orders) **WRITE** INSERT Order + OrderItems

Gerar	orderId = UUID
Definir	Order.status = PENDING
Associar	Order.seatLockId = lockId obtido no F4
Persistir	1 OrderItem por item do CheckoutRequest com appliedPrice calculado

Request: Order { userId, eventId, seatLockId, totalAmount, status: PENDING, items: [...] }

Response: orderId gerado + OrderSummaryResponse com paymentUrl para o frontend

Nota: O `paymentUrl` pode ser um link para a tela de pagamento do frontend com o `orderId` como parâmetro. Manter Order em `PENDING` até o Payment confirmar.

Casos de Erro

Lock expirado	HTTP 410 Gone — instruir cliente a refazer a seleção
Lock de outro usuário	HTTP 403 Forbidden
Setor indisponível	HTTP 422 — <code>EventSectorPricing.isAvailable == false</code>
TicketType inválido	HTTP 400 Bad Request

F6**Processamento de pagamento**

Integração com gateway externo e atualização de status coordenada

Payment

O Payment Service desacopla a lógica financeira dos demais serviços. Valida o pedido e o lock antes de cobrar, integra com o gateway externo (Stripe/Mercado Pago/PIX) e coordena a atualização de status entre Checkout e Inventory.

Passo 1 Cliente submete intenção de pagamento

Frontend

→ **Payment Service (via Gateway)****POST**

POST /payments/intent

Requer	JWT válido — userId do token
Validar	provider é um valor válido: STRIPE, MERCADO_PAGO, PIX
Validar	paymentMethodToken presente (exceto PIX que gera QR Code)

Request: `PaymentIntentRequest { orderId, provider, paymentMethodToken }`**Response:** `PaymentResultResponse { transactionId, status: PENDING, qrCodePix (se PIX) }`**Passo 2** Validar pedido no Checkout Service

Payment Service

→ **Checkout Service****GET**

GET /orders/{orderId}

Validar	Order existe
Validar	Order.userId == userId do JWT
Validar	Order.status == PENDING ou AWAITING_PAYMENT
Obter	totalAmount para confirmar valor a cobrar

Request: `Path param: orderId`**Response:** `Order` completo com `totalAmount` e `seatLockId`**Passo 3** Validar lock ainda ativo

Payment Service

→ **Inventory Service****GET**

GET /inventory/locks/{lockId}/status

Validar	Lock ainda existe no Redis (TTL não expirou)
Validar	lock.status == LOCKED
Crítico	Se lock expirou: cancelar pedido, não cobrar o cliente

Request: `Path param: lockId` (obtido do `Order.seatLockId`)**Response:** `SeatStatusResponse { lockId, status, expiresAt }`**Passo 4** Atualizar Order para AWAITING_PAYMENT

Payment Service

→ **Checkout Service****PATCH**

PATCH /orders/{orderId}/status

Ação	Mudar Order.status de PENDING para AWAITING_PAYMENT
Objetivo	Evitar que outros pagamentos sejam iniciados para o mesmo pedido

Request: `{ status: AWAITING_PAYMENT }`**Response:** HTTP 200**Passo 5** Processar cobrança no gateway externo

Payment Service

→ **Gateway Externo (Stripe/MP/PIX)****POST**

API do provedor

Enviar	amount = Order.totalAmount, currency, paymentMethodToken
Receber	providerTransactionId, status da operadora
Persistir	Transaction { orderId, userId, amount, provider, providerTransactionId, status, processedAt }

Request: Payload específico do provedor (Stripe PaymentIntent, MP Preference, PIX Payload)

Response: TransactionStatus: APPROVED | REFUSED | ERROR

Passo 6 Coordenar resultado — caminho APROVADO

Payment Service → Checkout Service **PATCH** PATCH /orders/{orderId}/status

Ação	Order.status = COMPLETED
------	--------------------------

Request: { status: COMPLETED }

Response: HTTP 200

Passo 7 Confirmar lock como SOLD no Inventory

Payment Service → Inventory Service **PATCH** PATCH /inventory/locks/{lockId}/confirm

Ação	SeatLock.status = SOLD
Ação	Remover TTL — registro não deve expirar mais
Opcional	Mover para base relacional de histórico consolidado

Request: Path param: lockId

Response: HTTP 200

Passo 8 Acionar Ticket Service para emitir ingresso

Payment Service → Ticket Service **POST** POST /tickets/generate

Enviar	orderId para o Ticket Service buscar os dados completos
Nota	Este acoplamento síncrono será substituído por evento RabbitMQ na fase final

Request: { orderId }

Response: { ticketId, qrCodeHash }

Nota: CAMINHO RECUSADO: se o gateway retornar REFUSED, Payment atualiza Order para CANCELED e chama /inventory/locks/{lockId}/release para liberar o assento.

Casos de Erro

Lock expirado antes do pagamento	HTTP 410 — não cobrar, cancelar pedido
Pagamento recusado	Order → CANCELED, SeatLock → AVAILABLE
Timeout no gateway externo	Manter Transaction com status ERROR, retentar ou notificar
Order já pago (idempotência)	HTTP 409 — verificar status antes de processar

F7**Emissão do ingresso digital**

Geração do QR Code após pagamento aprovado

Ticket

O Ticket Service gera o ingresso digital com um hash dinâmico anti-cambismo. Cada ingresso tem um QR Code único vinculado ao titular. O hash deve ser temporal (renovável) para dificultar transferência não autorizada.

Passo 1 Obter dados completos do pedido

Ticket Service

→ Checkout Service

GET

GET /orders/{orderId}

Validar	Order.status == COMPLETED
Obter	userId, eventId, items (sectorId, ticketType, seatNumber por item)

Request: Path param: orderId

Response: Order { userId, eventId, items, totalAmount, seatLockId }

Passo 2 Obter dados do evento para o ingresso

Ticket Service

→ Event Catalog

GET

GET /events/{eventId}

Obter	title, eventId, venue.name, venue.city para compor o ingresso impresso
-------	--

Request: Path param: eventId

Response: EventDetailsResponse

Passo 3 Obter nome do titular

Ticket Service

→ Identity Service

GET

GET /users/{userId}/profile

Obter	fullName para personalizar o ingresso
Alternativa	Extrair do payload do JWT (se disponível no contexto da requisição)

Request: Path param: userId

Response: { userId, fullName, email }

Passo 4 Gerar hash do QR Code e persistir ingresso

Ticket Service

→ PostgreSQL (Tickets)

WRITE

INSERT Ticket

Gerar	qrCodeHash = HMAC-SHA256(ticketId + userId + eventId + timestamp_expiry, secret)
Definir	Ticket.status = VALID
Persistir	1 Ticket por OrderItem (1 ingresso por assento)

Request: Ticket { orderId, eventId, userId, sectorId, seatNumber, qrCodeHash, status: VALID }

Response: ticketId gerado

Nota: Gerar 1 Ticket por OrderItem. Se o pedido tem 2 ingressos, persistir 2 registros Ticket distintos com QR Codes únicos.

Casos de Erro

Order não COMPLETED	HTTP 422 — não emitir ingresso
Ticket já gerado para o orderId	Retornar idempotente — não duplicar
Falha ao buscar dados do Event Catalog	Retentar com backoff antes de falhar

F8**Validação de acesso na catraca**

Autenticação física do ingresso no dia do evento

Ticket

Fluxo de altíssima criticidade e baixa latência. O operador da catraca (GATE_OPERATOR) escaneia o QR Code e recebe resposta imediata. Não deve depender de outros serviços no caminho crítico — toda a lógica está no Ticket Service + seu próprio banco.

Passo 1 Catraca envia QR Code para validação

Dispositivo de Catraca → Ticket Service (via Gateway) **POST** `POST /tickets/validate`

Requer	JWT com role == GATE_OPERATOR
Validar	gateId corresponde a um ponto de acesso válido

Request: `ValidateAccessRequest { qrCodeHash: string, gateId: string }`

Response: `AccessValidationResponse { isAllowed, result, message, sectorName, seatNumber }`

Passo 2 Buscar e validar o ingresso

Ticket Service → PostgreSQL (Tickets) **GET** `SELECT Ticket WHERE qrCodeHash = ?`

Validar	qrCodeHash existe no banco
Validar	Ticket.status == VALID → permite entrada
Checar	Ticket.status == USED → negar (DENIED_USED)
Checar	Ticket.status == REVOKED → negar (DENIED_REVOKED)
Checar	Hash não encontrado → negar (DENIED_INVALID)
Opcional	Verificar se hash não expirou (se implementar rotação de hash)

Request: N/A – consulta interna ao banco

Response: `Ticket { ticketId, status, sectorId, seatNumber, eventId }`

Passo 3 Registrar log de acesso

Ticket Service → PostgreSQL (Tickets) **WRITE** `INSERT AccessLog`

Persistir	<code>AccessLog { ticketId, gateId, accessedAt, result }</code>
Sempre	Registrar log independente do resultado (GRANTED ou DENIED)

Request: `AccessLog { ticketId, gateId, accessedAt: now(), result: AccessResult }`

Response: N/A – escrita interna

Passo 4 Marcar ingresso como USED (se entrada concedida)

Ticket Service → PostgreSQL (Tickets) **PATCH** `UPDATE Ticket SET status = USED`

Atômico	UPDATE com WHERE status = VALID para evitar double-entry
Se UPDATE retornar 0 rows	Outro processo já marcou como USED → negar entrada

Request: `UPDATE Ticket SET status='USED' WHERE ticketId=? AND status='VALID'`

Response: `rows_affected: 1 (sucesso) | 0 (já usado)`

Nota: SLA esperado: < 500ms. Zero chamadas a outros serviços no caminho crítico. O gateld deve ser pré-registrado e validado no próprio Ticket Service.

Casos de Erro

DENIED_USED	Ingresso já utilizado — possível tentativa de reuso
DENIED_REVOKED	Ingresso cancelado — pedido foi estornado
DENIED_INVALID	QR Code não reconhecido ou adulterado
Timeout de BD	Retornar erro genérico e logar — não liberar a catraca em caso de dúvida

F9

Cancelamento de pedido e estorno

Fluxo de compensação — ordem das operações é crítica

Checkout/Order

Cancelamentos podem ocorrer antes ou após o pagamento. A ordem de operações importa para garantir consistência: revogar ingresso → liberar assento → processar estorno → atualizar pedido.

Passo 1 Solicitar cancelamento

Frontend / Admin → Checkout Service (via Gateway) **PATCH** `PATCH /orders/{orderId}/cancel`

Validar	Order.userId == userId do JWT (ou role == ADMIN)
Validar	Order.status não é CANCELED nem REFUNDED (idempotência)
Regra	PENDING: cancelamento livre. AWAITING_PAYMENT: verificar política. COMPLETED: exige estorno

Request: Path param: `orderId` | Opcional: { `reason`: string }

Response: HTTP 200 { `orderId`, `status`: CANCELED | REFUNDED }

Passo 2 Revogar ingresso (se já emitido)

Checkout Service → Ticket Service **PATCH** `PATCH /tickets/revoke-by-order/{orderId}`

Condicional	Executar apenas se Order.status == COMPLETED e tickets foram emitidos
Ação	Ticket.status = REVOKED para todos os tickets do orderId
Primeiro	Revogar ANTES de liberar o assento — evita reuso durante a janela de cancelamento

Request: Path param: `orderId`

Response: HTTP 200 { `revokedCount`: int }

Passo 3 Processar estorno financeiro (se pagamento ocorreu)

Checkout Service → Payment Service **POST** `POST /payments/{transactionId}/refund`

Condicional	Apenas se Transaction.status == APPROVED
Enviar	transactionId da Transaction original
Receber	Confirmação do estorno do gateway externo
Persistir	Transaction atualizada com status CHARGEBACK

Request: { `transactionId`, `reason`: string }

Response: { `refundId`, `status`: CHARGEBACK }

Passo 4 Liberar assento no Inventory

Checkout Service → Inventory Service **PATCH** `PATCH /inventory/locks/{lockId}/release`

Ação	Remover SeatLock do Redis (repository.delete)
Se SOLD	Recriar SeatLock com status AVAILABLE e sem TTL
Após	Assento fica disponível para nova compra

Request: Path param: `lockId` (Order.seatLockId)

Response: HTTP 200

Passo 5 Atualizar status final do pedido

Checkout Service → PostgreSQL (Orders) PATCH UPDATE Order SET status = ?

CANCELED	Se não houve pagamento confirmado
REFUNDED	Se houve pagamento e estorno foi processado

Request: Order.status = CANCELED | REFUNDED

Response: HTTP 200

Nota: Ordem obrigatória: 1 Revogar ticket → 2 Estornar → 3 Liberar assento → 4 Atualizar Order. Inverter cria janelas de inconsistência.

Casos de Erro

Estorno recusado pelo gateway	Manter Order como COMPLETED, escalar para operação manual
Ticket já REVOKED	Ignorar — idempotente
Lock já expirado	Ignorar step 4 — assento já foi liberado pelo TTL

F1

Roteamento e controle de acesso

Entry point único — validação de JWT e roteamento por role

API Gateway

O Spring Cloud Gateway é o único ponto de entrada externo do sistema. Consulta o Netflix Eureka para descobrir instâncias saudáveis e aplica load balancing. Valida o JWT antes de rotear para serviços protegidos.

Passo 1 Receber requisição do cliente

Frontend / Mobile

→ API Gateway

ALL

<https://api.matchpass.com/>*

Extrair	Authorization: Bearer do header
Rotas públicas	GET /events/**, POST /auth/login, POST /users/register → bypass de validação

Request: Qualquer método HTTP + path

Response: N/A — intermediário

Passo 2 Validar JWT em rotas protegidas

API Gateway

→ Identity Service

GET

[GET /auth/validate](#)

Validar	Assinatura, expiração e estrutura do JWT
Extrair	userId, role do payload
Injetar	Headers X-User-Id e X-User-Role para os serviços downstream

Request: Header: Authorization: Bearer {jwt}

Response: HTTP 200 + { userId, role } | HTTP 401

Passo 3 Verificar autorização por role

API Gateway

→ Regras de roteamento

GET

[Eureka: resolução de rota](#)

CUSTOMER	Acesso a: /orders/**, /payments/**, /tickets/{id} (próprio), /inventory/**
GATE_OPERATOR	Acesso exclusivo a: POST /tickets/validate
ADMIN	Acesso total a todos os endpoints
Sem role adequada	HTTP 403 Forbidden

Request: X-User-Role header + path da requisição

Response: Permissão concedida ou HTTP 403

Passo 4 Rotear para o serviço destino via Eureka

API Gateway

→ Serviço destino

PROXY

[lb://{service-name}/path](#)

Consultar	Eureka para obter instâncias saudáveis do serviço
Load balancer	Round-robin entre instâncias disponíveis
Repassar	Headers originais + X-User-Id + X-User-Role

Request: Requisição original com headers enriquecidos

Response: Resposta do serviço destino repassada ao cliente

Nota: Rate limiting recomendado: 100 req/min por IP em rotas públicas, 300 req/min por userId em rotas autenticadas. Configurar via filtros do Spring Cloud Gateway.

Casos de Erro

JWT ausente em rota protegida	HTTP 401 Unauthorized
JWT expirado	HTTP 401 — cliente deve renovar via /auth/refresh
Role sem permissão	HTTP 403 Forbidden
Serviço indisponível no Eureka	HTTP 503 Service Unavailable